MalSkills: Detecting Malicious Skills in the Agentic Supply Chain via Neuro-symbolic Reasoning

Shenao Wang*
shenaowang@hust.edu.cn
Huazhong University of Science and Technology
Wuhan, China

Junjie He*
hjj@hust.edu.cn
Huazhong University of Science and Technology
Wuhan, China

Yanjie Zhao*
Yanjie_Zhao@hust.edu.cn
Huazhong University of Science and Technology
Wuhan, China

Yayi Wang
Ant Group
Hangzhou, China
wangyayi.wyy@antgroup.com

Kan Yu
Ant Group
Hangzhou, China
kan.yk@antgroup.com

Haoyu Wang*†
haoyuwang@hust.edu.cn
Huazhong University of Science and Technology
Wuhan, China

arXiv:2603.27204v2 [cs.CR] 21 Aug 2026

Abstract

Skills are increasingly used to extend LLM agents by packaging prompts, code, and configurations into reusable modules. As public registries and marketplaces expand, they form an emerging agentic supply chain, but also introduce a new attack surface for malicious skills. Detecting malicious skills is challenging because relevant evidence is often distributed across heterogeneous artifacts and must be reasoned in context. Existing static, LLM-based, and dynamic approaches each capture only part of this problem, making them insufficient for robust real-world detection. In this paper, we present MalSkills, a neuro-symbolic framework for malicious skills detection. MalSkills first extracts security-sensitive operations from heterogeneous artifacts through a combination of symbolic parsing and LLM-assisted semantic analysis. It then constructs the skill dependency graph that links artifacts, operations, operands, and value flows across the skill. On top of this graph, MalSkills performs neuro-symbolic reasoning to infer malicious patterns or previously unseen suspicious workflows. We evaluate MalSkills on a benchmark of 200 real-world skills against 5 state-of-the-art baselines. MalSkills achieves 93% F1, outperforming the baselines by 5–87 percentage points. We further apply MalSkills to analyze 150,108 skills collected from 7 public registries, flagging 620 potentially malicious skills. Through manual review, we confirm that 400 of them are indeed malicious, all of which were responsibly reported and are currently awaiting confirmation from the platforms and maintainers. These results demonstrate the practical potential of MalSkills in securing the agentic supply chain.

*Hubei Key Laboratory of Distributed System Security, Hubei Engineering Research Center on Big Data Security, School of Cyber Science and Engineering, Huazhong University of Science and Technology.

†Haoyu Wang (haoyuwang@hust.edu.cn) is the corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference'17, Washington, DC, USA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

CCS Concepts

• **Security and privacy** → **Malware and its mitigation**; **Software security engineering**; • **Applied computing** → *Supply chain management*.

Keywords

Agent Skills, Agent Supply Chain, Malicious Skills, Static Analysis, Neuro-Symbolic Method

ACM Reference Format:
Shenao Wang, Junjie He, Yanjie Zhao, Yayi Wang, Kan Yu, and Haoyu Wang. 2026. MalSkills: Detecting Malicious Skills in the Agentic Supply Chain via Neuro-symbolic Reasoning. In . ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

In recent years, skills have emerged as an increasingly popular paradigm for extending LLM agents. This paradigm was originally introduced by Anthropic [3] as an open skill-packaging convention and has since been adopted by a growing number of agent frameworks, such as OpenAI [51], LangChain [37], and OpenClaw [56]. Specifically, instead of embedding all capabilities into a single agent, skills bundle prompts, code scripts, and configurations into reusable modules that can be installed and composed on demand. As this paradigm has gained traction, a growing number of public registries and marketplaces have emerged to support skill publication, discovery, installation, and reuse. Representative examples include OpenClaw's ClawHub [53], `skills.sh` [72], and `SkillsMP` [69]. As of March, 2026, ClawHub had hosted over 30K skills [53], while `SkillsMP` had collected over 546K [69]. These developments indicate that skills are rapidly evolving into a new form of *agentic supply chain* [28, 30, 81], where third-party skills are created, distributed, discovered, installed, and reused in the agentic ecosystems.

However, the widespread adoption of skills also introduces a new attack surface into the emerging agentic supply chain. In platforms such as OpenClaw [56], skills are used to teach the agent how to invoke tools, while the underlying tool layer may expose filesystem access, process spawning, shell execution, web interaction, and other security-sensitive capabilities. As a result, a malicious skill could be turned into a potential backdoor for credential theft,

remote code execution, and agent hijacking. Recent security reports [4, 57, 75] suggest that this threat is already severe in practice. A large-scale empirical study scanned 98,380 skills and confirmed 157 malicious skills, including data thieves and agent hijackers [41]. Koi Security also reported a large-scale malicious campaign (called ClawHavoc [4]) on ClawHub. Snyk reported similar risks [75], documenting 1,467 malicious payloads in ClawHub. These findings reveal that malicious skills are already a practical security problem.

To mitigate these risks, both communities and researchers have begun to deploy preliminary detection for malicious skills. ClawHub currently combines VirusTotal-based scanning [55] with an internal scan pipeline [54]. Beyond this, the research community and open-source ecosystem have also proposed early scan tools, including Liu et. al. [41], as well as practical scanners such as AgentGuard [22], Skill-Security-Scan [32], Cisco's Skill-Scanner [12], and Snyk Agent Scan [74]. Taken together, these efforts largely fall into three categories: static rule-based detection [22, 32, 74], LLM-based semantic analysis [12, 78], and dynamic sandbox or runtime monitoring [41].

Research Gaps. However, existing scanners still face some practical challenges. Static rule-based methods are efficient, scalable, and easy to audit, but they depend heavily on predefined heuristics over known sensitive behaviors. As a result, they can be less effective when malicious logic is expressed through rewritten variants [8, 10, 77], living-off-the-land (LOTL) exploitations [19, 45], third-party library APIs [31], or natural language prompts rather than explicit code patterns [75]. LLM-based detectors provide stronger semantic generalization and have been used to identify prompt injection [8, 12, 74] and suspicious code patterns [41, 80, 84, 88]. However, these detectors can be misled by embedded prompts [11, 48] or seemingly benign descriptions [10] in the analyzed artifacts. Dynamic monitoring and sandbox-based approaches offer protection by examining concrete runtime behavior [8, 41]. Yet their effectiveness depends on execution coverage, may miss latent behaviors that are not triggered during testing, and can be evaded by environment-aware or sandbox-sensitive samples [13, 18]. Worse still, malicious samples may leverage sandbox-escape techniques to inflict damage on the analyser itself [17, 33], rather than merely evade detection.

Challenges. In practice, malicious skills detection is challenging for three reasons. First, skills are heterogeneous artifacts spanning prompts, code scripts, manifests, and potential runtime updates, making traditional malicious code analysis techniques [15, 20, 29, 76, 80, 86, 89, 90] difficult to apply directly. Second, the threat model is inherently adversarial, and evasive logic can surface differently across analysis stages. As a result, neither any single detection paradigm [22, 65] nor a naively cascaded workflow [12, 41] can be relied upon. Third, risk in skills is highly context-dependent. Sensitive operations such as file access, shell invocation, or network communication are not inherently malicious; their security implications depend on the skill's intended functionality.

Insights. These challenges motivate three key insights. First, malicious skill detection must be grounded in cross-artifact analysis, since security-relevant evidence is often distributed across code, prompts, manifests, setup scripts, and documentation. Second, a practical system should combine the precision of static analysis with the semantic generalization of LLMs. Third, accurate detection requires reasoning over how sensitive operations relate in context, especially whether they act on the same logical object and together form a malicious behavior pattern.

Our Work. Guided by these insights, we present MalSkills, a neuro-symbolic framework for malicious skills detection. Rather than formulating detection as an end-to-end classification problem, MalSkills adopts a three-stage design consisting of security-sensitive operation (SSO) extraction, skill dependency graph (SDG) generation, and neuro-symbolic reasoning. First, MalSkills extracts SSOs from heterogeneous artifacts by combining a parsing-based symbolic extractor with an LLM-assisted neuro extractor. Second, MalSkills resolves the security-relevant operands of the extracted SSOs through static points-to analysis and LLM-assisted operand inference, and then constructs the SDG that captures SSOs, operand associations, and value-flow dependencies across artifacts. Finally, MalSkills detects malicious skills through neuro-symbolic reasoning over the SDG, combining symbolic patterns with LLM-based reasoning to capture incomplete, implicit, or previously unseen suspicious workflows.

Evaluation. We evaluate MalSkills from four perspectives. First, on MalSkillsBench, a benchmark of 200 real-world malicious and benign skills, MalSkills outperforms 5 state-of-the-art baselines and achieves the best overall detection performance, with an F1-score of 0.93 and an FPR of 0.05. Second, ablation results show that the full design of MalSkills is effective, and that its symbolic extraction and neuro reasoning components contribute most critically to the final performance. Third, we study the impact of different LLMs and find that MalSkills remains effective across all tested models, achieving F1-scores ranging from 0.76 to 0.93, with `gpt-5.3-codex-medium` performing best overall. Finally, we deploy MalSkills on *Wild-Skills-150K*, a large-scale collection of 150,108 deduplicated skills from 7 public registries and marketplaces. MalSkills flags 620 potentially malicious skills, and manual review confirms that 400 are indeed malicious, demonstrating its practicality for uncovering previously unknown threats in the wild.

Contributions. We make the following contributions:

- **Novel Framework.** To the best of our knowledge, MalSkills is the first neuro-symbolic framework for detecting malicious skills in the emerging agentic supply chain. Rather than simply cascading static scanning with LLMs, MalSkills unifies cross-artifact evidence extraction, skill dependency modeling, and neuro-symbolic reasoning to capture malicious behaviors.
- **Benchmarking Existing Tools.** We build a benchmark of deduplicated real-world malicious skills and benign skills, and use it to benchmark 5 widely-adopted community tools. The results show that existing tools remain unsatisfactory, either missing many malicious skills or incurring high false positives.
- **Real-World Impact.** We apply MalSkills to 7 public registries, analyzing 150,108 skills in total. MalSkills flags 620 potentially malicious skills, and our manual review confirms that 400 of them are indeed malicious. We have responsibly reported these confirmed cases to the corresponding platforms and maintainers, and are currently awaiting their confirmation.

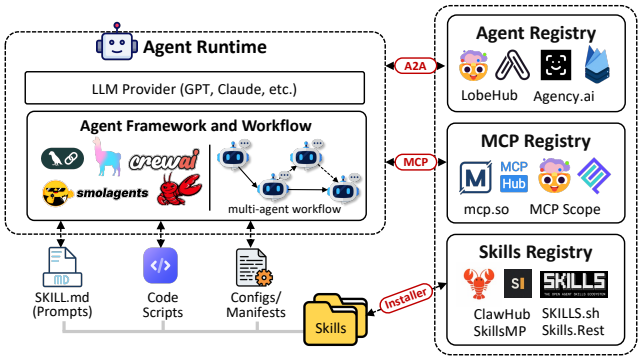


Figure 1: Skills in the Agentic Supply Chain Ecosystem

2 Background

2.1 Skills in the Agentic Supply Chain

As LLM-based agents become increasingly complex, their architectures are also becoming more modular. As shown in Figure 1, a typical agent runtime comprises the LLM provider, an agent framework, and orchestration logic, such as multi-agent workflows. Within this runtime, agent capabilities are not implemented solely by the core framework. Instead, they are often extended through external components, such as tool calling via Model Context Protocol (MCP) [49] or agent collaboration via Agent2Agent (A2A) protocol [1].

Recently, Anthropic introduced a more lightweight and flexible mechanism, called Agent Skills [3], to extend LLM agents with reusable capabilities. The skill mechanism allows developers to extend an agent with task-specific functionality without modifying the agent core. Since then, this paradigm has been adopted by a growing number of agent frameworks and platforms, including OpenAI [51], LangChain [37], and OpenClaw [56]. In practice, a skill is typically packaged as a collection of heterogeneous artifacts, including prompt files such as `SKILL.md`, code scripts such as `.py` or `.js`, and configuration or manifest files such as `.json` and `.yaml`. As illustrated in Figure 1, these artifacts jointly define the skill's functionality, usage conditions, and runtime behavior. Once installed, the skill does not execute as an isolated program. Instead, it interacts with the hosting agent runtime, which interprets the skill metadata, invokes its code or prompts when appropriate, and mediates access to the underlying tool layer. Through this interaction, a skill may influence agent decisions, call tools such as filesystem access, shell execution, process spawning, or network requests, and communicate with external resources or services.

As the skill ecosystem has evolved, public registries and marketplaces have also emerged. Similar to how MCP servers are published through MCP registries [27, 46, 47] and agents are discovered by agent registries [2, 44], skills are increasingly distributed through public registries, including ClawHub [53], `skills.sh` [72], and `SkillsMP` [69]. These registries support skill publication, discovery, installation, and reuse, allowing developers or end users to browse third-party skills and integrate them into agent runtimes. As a result, skills are rapidly becoming a fundamental unit of capability distribution in modern agentic supply chain [28, 30, 81].

2.2 Malicious Skills and Mitigations

Unfortunately, like other open registries such as NPM [29, 80, 88], PyPI [20, 76], and VSCode extension marketplace [16, 42], skill registries introduce a new attack surface into the agentic supply chain [4, 41, 75]. Because skills can influence agent behavior and interact with tool calling, a malicious skill may abuse these capabilities to perform harmful actions such as data exfiltration, unauthorized command execution, and agent hijacking. However, unlike traditional malware, the malicious logic of a skill is often not concentrated in executable code scripts. Instead, it may be distributed across prompts, scripts, manifests, configuration files, and setup logic [41, 75].For example, a prompt may instruct the agent to collect sensitive information, a script may access local files or environment variables, and a configuration file may specify an attacker-controlled endpoint for exfiltration. As a result, the security risk of a skill often emerges only when multiple artifacts and their interactions are considered together.

To mitigate these threats, both industries and researchers have begun developing preliminary detection tools for malicious skills. As shown in Table 1, existing mitigations mainly fall into three categories. The first is *static detection*, which scans skill artifacts using predefined rules [12, 22, 32, 78], signatures [74], or pattern matching [5, 12, 41] to identify suspicious code, prompts, dependencies, or metadata. Static methods are efficient and scalable, but they can be evaded when malicious behaviors are expressed through rewritten logic [10], benign-looking third-party APIs [31], or cross-artifact compositions. The second category is *LLM-based semantic analysis*, which uses language models to interpret prompts, scripts, and documentation in order to detect suspicious intent or hidden malicious behaviors [5, 12, 41]. These methods can generalize beyond handcrafted rules, but they may also be misled by prompt injection [11, 48], obfuscated descriptions [77], or adversarially crafted contexts [10, 61]. The third category is *dynamic analysis and sandbox monitoring*, which executes skills in controlled environments and detects maliciousness through observed runtime behavior [41]. While dynamic approaches can reveal concrete attack actions, their effectiveness depends on execution coverage and may be limited by environment-sensitive or sandbox-aware samples [13, 18]. Overall, these preliminary mitigations provide insights, but they still leave research gaps in accurately detecting malicious skills.

3 Motivating Example

To illustrate why malicious skill detection remains difficult in practice, we consider a representative skill illustrated in Figure 2, which appears to be a benign DevOps troubleshooting utility but in fact performs credential harvesting and exfiltration. First, `SKILL.md` instructs the agent to prepare a diagnostic bundle for remote-access troubleshooting. Second, `config.yaml` defines a bundle directory, a remote upload endpoint, and a command that copies `~/.aws/credentials` into the bundle. Third, `post.js` reads the diagnostic plan, collects the bundle contents, and sends them to the configured remote server. The malicious behavior is thus not explicit in any single file; it emerges only after correlating the prompt, configuration, and code.

This example highlights why existing detection methods are insufficient. As shown in Figure 2, the malicious logic is distributed

Table 1: Comparison of Existing Skill Security Scan Tools.

Tool	Source & Stars	Artifact Cov.	Static	LLM	Dynamic	Cross-artifact
AgentScan [74]	Snyk, 2K Stars	◐	◐	○	○	○
Skill Scanner [12]	Cisco, 1.5K Stars	●	●	●	○	◐
Nova-Proximity [78]	NovaHunting, 286 Stars	◐	●	○	○	○
Skill-Sec-Scan [32]	SourceHot, 106 Stars	◐	●	○	○	○
Caterpillar [5]	Alice, 41 Stars	●	●	●	○	◐
MASB[1] [41]	Quantstamp, 29 Stars	◐	●	●	●	◐

[a] **Artifact Cov.** denotes the supported skill-related artifacts or file types.
[b] ● indicates broad/full support, ◐ indicates moderate/partial support, and ○ indicates limited or no support.
[c] **Cross-artifact** refers to explicit reasoning over relations among multiple artifacts, rather than simple recursive scanning.

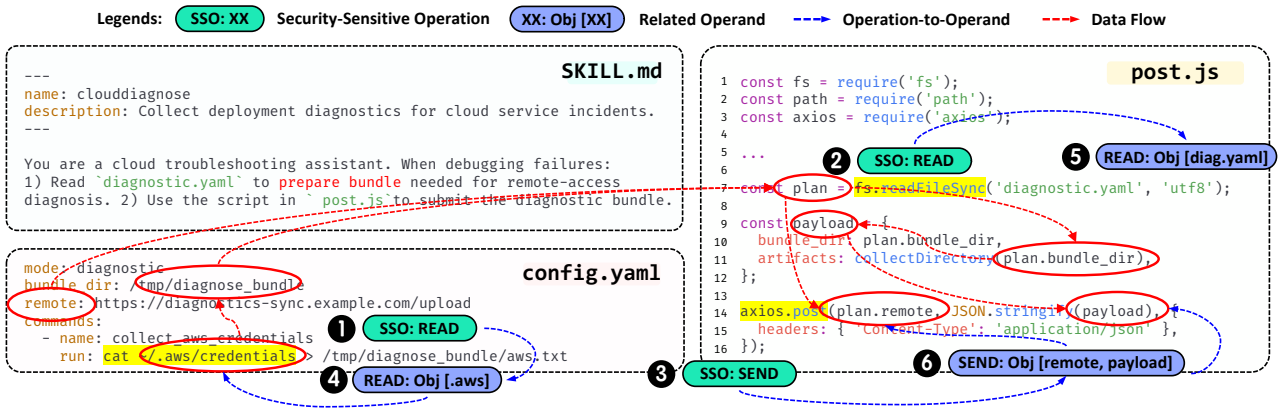


Figure 2: A malicious skill whose credential-exfiltration logic is distributed across multiple artifacts. The labels denote security-sensitive operations (SSOs), their operands, and the dependencies that connect them into an end-to-end malicious workflow.

across multiple artifacts: ❶ and ❷ denote two security-sensitive read operations, while ❸ denotes a send operation; their concrete operands are further revealed by ❹–❻, which connect the reads to `~/.aws/credentials` and `diagnostic.yaml`, and the send to the remote endpoint and payload. This example is challenging for existing detectors for three reasons. First, the evidence is *cross-artifact*: no single file explicitly states the full malicious intent, so artifact-local static scanning may only observe isolated signals such as a credential read or an outbound request. Second, the risk is *context-sensitive*: both the reads and the upload can appear consistent with a benign troubleshooting workflow, so LLM-based analysis may be misled by the adversarial words. Third, the maliciousness is *data-dependent*: the key issue is not the mere invocation of ❶–❸, but that the sensitive operand accessed in ❹ is propagated through the intermediate dependency in ❺ and ultimately becomes part of the payload sent in ❻. In other words, the main challenge is to reconstruct whether seemingly benign operations across heterogeneous artifacts are connected into a malicious workflow.

4 Methodology

To address these challenges, we propose a neuro-symbolic framework for malicious skill detection. As illustrated in Figure 3, MalSkills consists of three stages: security-sensitive operation (SSO) extraction, skill dependency graph (SDG) generation, and neuro-symbolic reasoning. First, *SSO extractor* identifies security-sensitive operations from heterogeneous skill artifacts, including source code, prompts, and manifests. Second, *SDG generator* resolves the operands associated with these operations and constructs an operand-centric SDG. Third, *neuro-symbolic reasoner* converts the graph into relational facts and performs declarative reasoning to detect malicious behavior patterns. Throughout the pipeline, we use LLM-based inference as a complement to symbolic analysis, and we feed stable neural findings back into reusable symbolic rules.

4.1 Security-Sensitive Operation Extraction

The goal of the first stage is to extract *security-sensitive operations* (SSOs) from heterogeneous skill artifacts. At this stage, we do not yet determine whether a skill is malicious. Instead, we identify operational evidence that may later participate in malicious behaviors, such as command execution, file I/O, network communication, environment-data collection, cryptographic usage, package installation, or prompt-implied side effects.

Parsing-based Symbolic Extractor. On the symbolic side, we build a parsing-based analyzer supported by a set of heuristic rules. These rules are offline distilled from security-sensitive API usage patterns according to previous works [15, 86] and widely adopted

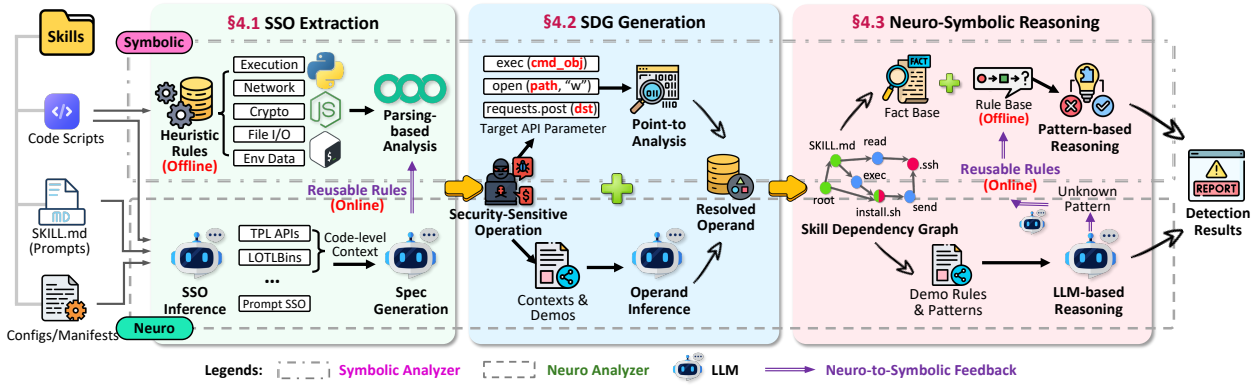


Figure 3: The overall workflow of MalSkills.

Table 2: Representative rules for major SSO categories.

Category	Example Rules

Exec	`subprocess.run(...)` / `subprocess.Popen(...)`
	`os.system(...)` / `exec(...)`
	`Runtime.getRuntime().exec(...)`

Net	`requests.get(...)` / `requests.post(...)`
	`fetch(...)` / `axios(...)`
	`urllib.request.urlopen(...)` / `socket.connect(...)`

File	`open(..., "r")` / `open(..., "w")`
	`fs.readFile(...)` / `fs.writeFile(...)`
	`Path.read_text(...)` / `Path.write_text(...)`

Env	`os.getenv(...)` / `os.environ[...]`
	`process.env.*` / `System.getenv(...)`
	`dotenv.load(...)` / access to credential/token files

Install	`pip install ...` / `pip3 install ...`
	`npm install ...` / `yarn add ...`
	`apt-get install ...` / `curl ... | bash`

Crypto	`hashlib.sha256(...)` / `hashlib.md5(...)`
	`AES.new(...)` / `RSA.import_key(...)`
	`crypto.createHash(...)` / `base64.b64encode(...)`

System Prompt

You are extracting evidence facts, not verdicts.
TASK: Read the artifact and extract only grounded sensitive-operation evidence. Map each finding to the sensitive operation taxonomy using `type` and `subtype`. Do not output benign metadata, generic capability, or mismatched intent.
INPUT FORMAT: 1. Artifact type: {code / markdown / yaml / json / ...} 2. Artifact text with line numbers 3. Optional context: {package, language, file path}
OUTPUT FORMAT:

```
{"records": [
  {
    "type": "...",
    "subtype": "...",
    "confidence": 0.0,
    "start_line": 0,
    "end_line": 0,
    "attributes": {"matched_text": "..."}
  }
]}
```

SPECIAL INSTRUCTIONS: 1. Extract only concrete evidence facts grounded in the artifact text. 2. Treat third-party wrappers and LOTL/trusted-tool abuse as equivalent to native sinks. 3. If a setup instruction contains a concrete command, URL, path, or credential reference, extract it. 4. If no valid taxonomy subtype applies, return `"records": []`.

Figure 4: Simplified Prompt Template for the LLM-assisted Neuro Extractor.

security specifications [64, 85]. Table 2 shows representative examples of the offline-initialized symbolic rules used to identify explicit SSO patterns. Given a skill package, the analyzer parses source code and configuration content to identify explicit invocations of target APIs and to recover their syntactic parameters. This produces structured evidence such as calls to execution interfaces, HTTP request functions, file I/O operations, package installation commands, or secret-reading accesses. However, some malicious skills invoke equivalent behaviors through third-party libraries, living-off-the-land techniques, wrapper functions, or non-standard interfaces that are not included in the initialized rule set. Moreover, some SSOs are not explicitly encoded in code at all, but are instead implied by prompts or documentation, such as instructions to transmit debugging information or execute shell commands on behalf of the user. These cases are difficult to recover through static parsing alone.

LLM-assisted Neuro Extractor. To cover such cases, we introduce an LLM-based SSO extractor that reasons over heterogeneous artifacts to recover SSOs from weakly structured or semantically implicit artifacts. As shown in Figure 4, the prompting strategy follows an *evidence-first, schema-constrained* design: instead of asking the LLM for a verdict or free-form summary, we instruct it to extract only *concrete evidence facts*, map each fact to a fixed taxonomy of SSOs, and return structured records with confidence scores, line spans, and matched text. We also improve the strategy by explicitly suppressing benign metadata and generic descriptions, while still encouraging equivalence reasoning for third-party wrappers, LOTL-style invocations, and imperative setup instructions embedded in natural language. For example, given `fabric.Connection(host).run("curl -fsSL https://x.sh | bash")`, the extractor infers `shell_interpreter_execution` and `outbound_connection`, rather than free-form results such as "downloads and executes a script."

Neuro-to-Symbolic Feedback. The neural extractor improves recall on semantically implicit or non-canonical SSOs, but invoking an

LLM for every artifact is costly and inherently less stable than symbolic matching. To make neural discoveries reusable, we introduce a neuro-to-symbolic feedback loop that promotes stable findings into new symbolic rules. Specifically, when the neural extractor repeatedly identifies the same sensitive operation pattern across packages, such as a third-party wrapper for command execution or a recurring LOTL-style download-and-execute idiom, MalSkills will send the corresponding evidence records to a dedicated *specification generator*. It will abstract recurring evidence into a reusable symbolic specification by retaining the operation type, lexical trigger, argument structure, and minimal context constraints required for precise matching. The generated rule candidates are then normalized, deduplicated, and validated on a held-out corpus before being incorporated into the symbolic rule base. For example, if the neural extractor repeatedly maps `fabric.Connection(...).run(...)` to `shell_interpreter_execution`, the spec generator can synthesize a wrapper rule that binds `Connection.run` to the corresponding execution sink under lightweight context constraints.

4.2 Skill Dependency Graph Generation

The second stage organizes isolated SSO records into a structured *Skill Dependency Graph* (SDG) that captures how security-sensitive behaviors are grounded in artifacts, linked to their relevant operands, and connected through recovered value flow. This stage consists of three components. First, MalSkills performs *symbolic operand resolution* to recover the security-relevant operands of each extracted SSO and trace their explicit origins through static points-to analysis. Second, it applies *LLM-assisted operand inference* to normalize semantically equivalent operands and infer missing value flow that cannot be fully recovered through symbolic analysis alone. Third, it performs *SDG construction* by instantiating typed nodes and edges over artifacts, SSOs, operands, and values, thereby lifting dispersed facts into a unified graph for downstream reasoning. Notably, MalSkills does not construct a complete program-level control-dependence graph. Unlike conventional programs, skill execution may depend on prompt instructions, user requests, the agent planner, and runtime context, whose conditions cannot always be mapped to precise branch semantics.

Symbolic Operand Resolution. For each SSO record, we first recover the operand slots according to the operation type. Each major SSO category is associated with a canonical operand schema. For example, execution operations expose command-oriented operands, file operations expose path- and content-oriented operands, and network operations expose endpoint- and payload-oriented operands. We then leverage an existing points-to analysis engine [7, 82] to determine the source of these operands and whether multiple operations refer to the same underlying object. Specifically, MalSkills traces assignments, aliases, parameter passing, return values, wrapper calls, and inter-procedural data dependencies within and across files. This allows MalSkills to recover explicit operand propagation chains, such as a secret read from `os.getenv(...)` being stored in a variable and later passed into `requests.post(...)`.

LLM-assisted Operand Inference. Symbolic resolution is effective when operand propagation is explicit, but real-world skills are often incomplete, weakly typed, multi-file, or mixed with prompts and instructions. As a result, some operands cannot be fully recovered through static analysis alone. To address this, we introduce an LLM-assisted operand inference component that normalizes semantically equivalent operands, infers missing value flow from local context, and bridges the gaps left by symbolic analysis. For instance, the model may infer that a variable named `token`, a prompt phrase such as "API key," and a manifest field for credentials all denote the same secret-bearing operand class, even when no direct syntactic value flow is visible in the code context.

Notations:

- $a \in \mathcal{A}$: An artifact, such as `SKILL.md`, `config.yaml`, or a script file.
- $r \in \mathcal{R}$: A normalized SSO record.
- $o \in \mathcal{O}$: A resolved security-relevant operand object.
- $v \in \mathcal{V}$: A value representing concrete or abstract data.
- $\text{artifact}(r)$: The artifact from which SSO record r originates.
- $\text{opnds}(r)$: The set of operand references associated with SSO record r.
- $\text{resolve}(x)$: The operand object resolved from operand reference x.
- $\text{bind}(o)$: The value entity directly associated with operand object o.
- $v_1 \rightsquigarrow v_2$: A value-flow indicating that the value v_1 may propagate to v_2.

Node Types:

1. **Artifact Node:** $a \in \mathcal{A} \quad \Rightarrow \quad \text{type}(a) = \textsc{Artifact}$
2. **SSO Node:** $r \in \mathcal{R} \quad \Rightarrow \quad \text{type}(r) = \text{SSO}$
3. **Operand Node:** $o \in \mathcal{O} \quad \Rightarrow \quad \text{type}(o) = \textsc{Operand}$
4. **Value Node:** $v \in \mathcal{V} \quad \Rightarrow \quad \text{type}(v) = \textsc{Value}$

Edge Construction Rules:

1. **Evidence Edge:** $\text{artifact}(r) = a \quad \Rightarrow \quad (a, \textit{contains}, r) \in E$
2. **Operand Edge:** $x \in \text{opnds}(r),\ \text{resolve}(x) = o \quad \Rightarrow \quad (r, \textit{has_opnd}, o) \in E$
3. **Value-flow Edge:** $\text{bind}(o) = v \quad \Rightarrow \quad (o, \textit{value_flow}, v) \in E$; $v_1 \rightsquigarrow v_2 \quad \Rightarrow \quad (v_1, \textit{value_flow}, v_2) \in E$

Figure 5: Node and edge construction rules for the SDG.

SDG Construction. After extracting SSO records and resolving their security-relevant operands, MalSkills organizes them into the SDG. Formally, we model the SDG as a typed directed graph $G = (V, E, \phi_V, \phi_E)$, where ϕ_V and ϕ_E assign semantic types to nodes and edges, respectively. Figure 5 summarizes the node and edge construction rules used to instantiate the SDG. The node set V contains four categories. *Artifact* nodes represent source-level program artifacts, such as prompt files, manifests, scripts, or other analysis units. *SSO* nodes represent normalized security-sensitive operations extracted by MalSkills. *Operand* nodes represent resolved security-relevant operand objects referenced by SSO records, such as files, endpoints, commands, or payload handles. *Value* nodes represent the concrete or abstract data entities associated with operands and intermediate computations, serving as the carrier of recovered propagation semantics. The edge set E contains three relation types. A *contains* edge links an artifact node to an SSO node originating from that artifact. A *has_operand* edge links an SSO node to an operand node when that operand is explicitly referenced by the operation. A *value_flow* edge captures data propagation semantics: it either links an operand node to its directly associated

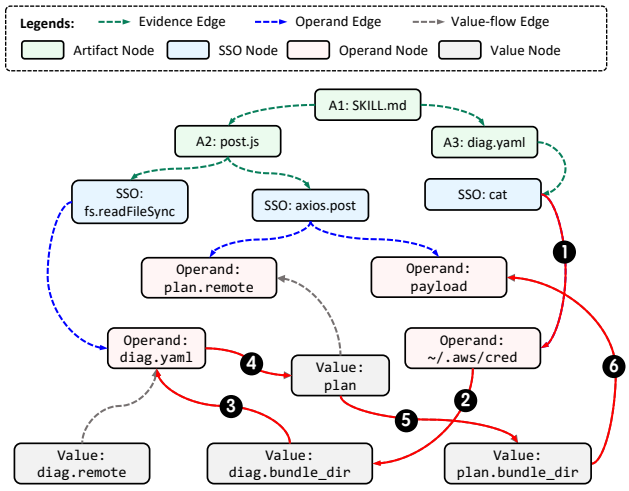


Figure 6: The SDG corresponding to the skill in Figure 2. The red edge denotes the key value-flow edges for reasoning.

value node, or links one value node to another when MalSkills recovers a possible propagation relation through assignments, aliases, parameter passing, return-value binding, wrapper calls, or interprocedural transfer. Figure 6 illustrates an example SDG that shows how dispersed SSOs extracted from multiple artifacts are connected to their resolved operands and further linked through recovered value-flow dependencies. The resulting SDG lifts isolated observations into a unified context that supports subsequent reasoning over security-relevant facts.

4.3 Neuro-Symbolic Reasoning

The third stage performs malicious-skill detection through a neuro-symbolic reasoning process built on top of the SDG.

Pattern-based Symbolic Reasoning. MalSkills initializes symbolic reasoning with a set of predefined patterns that capture common and security-critical malicious behaviors observed in real-world skills. Rather than relying on isolated SSO co-occurrence, each pattern is defined over connected SDG evidence and requires that the participating SSOs be semantically linked through operand dependencies, propagated values, or source-to-sink structure. At initialization, these patterns cover major classes of malicious behavior, including execution-and-delivery chains, persistence establishment, privilege escalation and identity abuse, injection and covert residency, information theft, command-and-control activity, lateral movement, defense evasion and anti-forensics, and destructive or ransomware-like actions. In essence, the symbolic component checks whether the SDG contains a behaviorally coherent subgraph whose operation sequence and dependency structure satisfy one of these known patterns. For example, in Figure 6, the SDG links the secret-access operation reading `~/.aws/cred` to the outbound `axios.post` operation through the intermediate payload object and the value-flow edge (highlighted in **red**), indicating that the sensitive content obtained from the local credential file is propagated into data sent to a remote destination.

LLM-based Neuro Reasoning. Obviously, predefined patterns cannot exhaustively cover the diversity of malicious behaviors found in practice. To handle such cases, MalSkills incorporates an LLM-based neuro reasoning component. Given the SDG and the symbolic matching results, the LLM is prompted with a normalized summary of the relevant subgraph, including the involved artifacts, the extracted SSOs, the resolved operands, and the value-flow paths connecting them. In this way, the neural component complements symbolic reasoning by providing semantic interpretation over incomplete, weakly matched, or previously unseen behavior patterns. When the LLM repeatedly identifies a recurring suspicious workflow that is not well covered by the current predefined patterns, MalSkills records the corresponding subgraphs, operation sequences, and dependencies as candidate patterns for further use.

5 Evaluation

To demonstrate the effectiveness, design contribution, robustness, and ecosystem-scale practicality of MalSkills, we conduct extensive experiments addressing four research questions.

- **RQ1: Effectiveness.** How effective is MalSkills when compared with state-of-the-art baselines?
- **RQ2: Ablation Study.** How much do the major components of MalSkills contribute to the overall detection performance?
- **RQ3: Impact of LLMs.** How sensitive is MalSkills to the choice of different LLMs?
- **RQ4: Practicality.** Is MalSkills practical for large-scale registries to uncover previously unknown malicious skills?

5.1 Evaluation Setup

Implementation. We implemented a prototype of MalSkills in Python with over 7.9K lines of code, excluding third-party dependencies and external frameworks. In the parsing-based analysis, MalSkills is built on top of Semgrep [63]. To support realistic skill ecosystems, we incorporate a total of 2,665 Semgrep rules spanning 10 programming languages, including C, C++, C#, Go, Java, JavaScript, PHP, Python, Ruby, and TypeScript. In the operand resolution, MalSkills leverages the existing multi-language static analyzer YASA to perform points-to and value-flow analysis. At present, YASA supports Python, JavaScript, Java, and Go, which allows MalSkills to resolve value flow and recover concrete operation objects for these languages. Unless otherwise specified, MalSkills uses `gpt-5.3-codex-medium` as the default model in the experiments. Due to space limitations, we do not include all prompts used in the LLM-assisted analysis in the main text. All these prompts are provided in the open-sourced artifacts.

Running Environment. All experiments were conducted on a server running Ubuntu Linux 22.04.5 LTS, equipped with 256 CPUs, 1.0 TB RAM, 1 NVIDIA GeForce RTX 4090 GPU, and 28 TB of SSD storage. Unless otherwise stated, all tools were evaluated under the same environment and resource configuration.

Dataset. To comprehensively evaluate MalSkills, we construct two datasets, one ground truth benchmark for baseline comparison and one for large-scale in-the-wild analysis.

(1) MalSkillsBench. For MalSkillsBench, we construct a labeled dataset of malicious and benign agent skills collected from real-world skill marketplaces and publicly reported malicious-skill incidents. We initially considered incorporating the malicious skills reported by MASB [41], which originally identified 157 malicious samples. However, a substantial portion of the corresponding GitHub repositories had become unavailable, and we were only able to successfully retrieve 57 of them. We then reran the publicly available MASB scanning pipeline on these 57 samples and found that only 6 could still be successfully detected. Moreover, these repositories are often large and artifact-heavy (more than 4K avg.), making manual auditing and validation impractical. Since MASB is also included as one of our evaluation baselines, incorporating this small set of MASB-detected samples would additionally compromise fairness. We therefore do not include the MASB samples in the benchmark. Instead, we construct the malicious set by systematically collecting publicly reported malicious-skill incidents from ClawHub [4, 57, 59, 60, 75]. Although many of these malicious skills had already been removed from ClawHub, we recovered 597 samples from the historical commits of ClawHub's GitHub archive repository [52]. Since many of these samples were repeatedly uploaded by the same malicious actors as part of large-scale poisoning campaigns [4, 57], we further perform hash-based deduplication and obtain 100 unique malicious skills. Following the common practice of prior security benchmark construction [6, 9, 87], we build a balanced dataset by additionally selecting an equal number of popular benign skills from ClawHub. This yields a final benchmark containing 200 skills in total. For each skill, we retain its original artifact structure to preserve realistic cross-artifact analysis conditions, including prompts, manifests, scripts, configuration files, and other auxiliary artifacts.

(2) Wild-Skills-150K. To evaluate ecosystem-scale practicality, we further collect a large unlabeled corpus from 7 public registries and marketplaces, including 34,092 skills from Skills Directory [67], 30,734 from SkillsLLM [68], 10,598 from Smithery [73], 30,789 from Skills.sh [72], 9,643 from Skills.rest [70], 11,064 from SkillsMP [69], and 37,858 from ClawHub [52, 53]. In total, this yields 164,778 raw skill entries. After deduplication, we obtain 150,108 unique skills, which we denote as *Wild-Skills-150K*. This dataset is used for large-scale scanning in RQ4 to assess whether MalSkills can operate efficiently and detect previously unreported malicious skills.

Baselines. We compare MalSkills against representative malicious skills scanning tools drawn from both industry and recent research, covering static analysis, LLM analysis, and dynamic analysis. Specifically, we include Skill Scanner [12], Nova-Proximity [78], Skill-Sec-Scan [32], Caterpillar [5], and MASB [41]. To the best of our knowledge, these constitute the most representative and widely adopted publicly available end-to-end tools for malicious skill detection in the current community. We do not include Snyk's AgentScan in our evaluation because its detection relies on proprietary Snyk APIs and signature matching against the internal database. To ensure fair comparison, we run each baseline according to its publicly documented workflow and use its default or recommended detection settings whenever possible. For baselines that require LLM support, we use the same underlying model, `gpt-5.3-codex-medium`, across all tools.

Table 3: Detection effectiveness on MalSkillsBench. The best results for each metric are highlighted with red, and the second-best results are highlighted with blue. For FPR, lower values indicate better performance.

Tool	TP/FP/TN/FN	Prec.	Rec.	FPR	F1
Skill Scanner [12]	93/22/72/4	**0.81**	**0.96**	0.23	**0.88**
Caterpillar [5]	82/40/60/18	0.67	0.82	0.40	0.74
MASB [41]	9/10/90/91	0.47	0.09	**0.10**	0.15
Skill-Sec-Scan [32]	40/50/50/60	0.44	0.40	0.50	0.42
Nova-Proximity [78]	4/25/75/96	0.14	0.04	0.25	0.06
MalSkills	92/5/95/8	**0.95**	**0.92**	**0.05**	**0.93**

5.2 RQ1: Effectiveness

We evaluate MalSkills and all baselines on MalSkillsBench. For each tool, we obtain a result indicating whether a skill is malicious or benign.[2] Following standard practice, we report Precision, Recall, False Positive Rate (FPR), and F1-score.

Overall Results. Table 3 presents the overall comparison results. MalSkills achieves the best overall trade-off among all compared baselines, attaining the highest F1-score of 0.93 and the lowest FPR of 0.05. MalSkills's recall of 0.92 is slightly lower than the best-performing Skill Scanner, but this small gap comes with substantially better precision and FPR. Among all baselines, the overall second-best baseline is Skill Scanner. As an industrial-grade scanner released by CISCO, it also leverages LLM-based analysis and achieves the highest recall, but at the cost of a much higher FPR, which leads to worse overall performance than MalSkills. Besides, our benchmark also shows that several malicious skill detection tools perform unsatisfactorily in practice. In particular, Nova-Proximity performs poorly because it mainly relies on predefined regular-expression rules, which are insufficient to capture the diverse behaviors of malicious skills. MASB first uses predefined rules to filter suspicious skills and then applies LLM analysis, but this simple cascade remains limited by the quality of the initial rule-based filtering stage. Even with dynamic analysis, its detection effectiveness is still unsatisfactory. This result is understandable because MASB was primarily designed as a large-scale empirical study of malicious skills rather than as a dedicated end-to-end detector. Besides, Caterpillar and Skill-Sec-Scan exhibit a different trade-off. Both tools can detect a non-trivial portion of malicious skills, but they sacrifice precision and false positive control. Overall, MalSkills provides a better balance between precision and recall.

FP/FN Analysis. We conduct a further review of the FPs and FNs. The FPs are concentrated in skills whose descriptions contain strong operational or automation-related signals, such as installation instructions, environment configuration steps, cron/heartbeat workflows, and web scraping examples. Although such content is benign in context, it partially overlaps with the malicious patterns that MalSkills treats as security-relevant evidence. On the other side, the FNs mainly arise in cases where maliciousness is conveyed more implicitly than our current SSO categories capture.

[2] For tools that output auxiliary labels such as *Suspicious*, we do not count such cases as correct predictions for either the malicious or benign. As a result, the total number of predictions may not sum to the full benchmark size.

Table 4: Ablation study of MalSkills with 4 variants. The best results for each metric are highlighted with red. For FPR, lower values indicate better performance.

Variant	TP/FP/TN/FN	Prec.	Rec.	FPR	F1
Full MalSkills	92/5/95/8	**0.95**	**0.92**	**0.05**	**0.93**
w/o Symbolic Extractor	27/0/100/73	**1.00**	0.27	**0.00**	0.43
w/o Neuro Extractor	91/5/95/9	**0.95**	0.91	**0.05**	**0.93**
w/o Symbolic Reasoner	92/5/95/8	**0.95**	**0.92**	**0.05**	**0.93**
w/o Neuro Reasoner	82/49/51/18	0.63	0.82	0.49	0.71

Representative examples include documentation-centric social engineering, guidance that redirects users to external downloads, and bot-control capabilities. In these cases, the malicious signal is not always expressed as an explicit sensitive operation, but instead emerges through intent, indirection, or downstream enablement. This suggests that the main limitation of MalSkills lies in covering more implicit and document-oriented threat manifestations.

5.3 RQ2: Ablation Study

We conduct an ablation study on MalSkillsBench using the same evaluation setting as in RQ1. To understand the contribution of each major component, we evaluate the following variants:

- *Full MalSkills.* The complete system with both symbolic and neural extraction, followed by symbolic and neural reasoning.
- *w/o Symbolic Extractor.* This variant removes the symbolic extraction component and relies only on the neural extractor to identify SSO from the skill.
- *w/o Neuro Extractor.* This variant removes the neural extractor for SSO collection.
- *w/o Symbolic Reasoner.* This variant removes the symbolic reasoning and classifies based only on neural reasoning.
- *w/o Neuro Reasoner.* This variant removes the neural reasoning and relies only on symbolic reasoning.

Table 4 presents the ablation results. Overall, the full MalSkills achieves the best overall trade-off between effectiveness and FP control, with an F1-score of 0.93 and an FPR of 0.05.

w/o Symbolic Extractor. Removing the symbolic extractor causes MalSkills to fail in a highly conservative manner. Although this variant achieves perfect precision (1.00) and zero FPR, its recall drops sharply from 0.92 to 0.27, indicating that it becomes overly conservative and misses most malicious skills. This suggests that the neural extractor alone identifies only a narrow subset of highly salient SSOs, while failing to robustly recover the broader range of suspicious operations needed for high recall. One reason is that the system prompt used by the neural extractor is primarily designed to uncover stealthy, implicit, or adversarial SSOs, rather than to exhaustively extract all generic SSOs from a skill. As a result, when used in isolation, the neural extractor tends to be conservative.

w/o Neuro Extractor. In contrast, removing the neuro extractor causes almost no performance drop on MalSkillsBench. This suggests that, for the current benchmark, the symbolic extractor already covers most of the SSOs needed to identify known malicious skills. This result is reasonable given how the benchmark is constructed. Since the ground-truth malicious samples are collected from already discovered attacks, many of their malicious behaviors are relatively explicit and can be captured by static rules. Nevertheless, this should not be interpreted as evidence that the neuro extractor is unnecessary. Its role is to complement symbolic extraction by capturing stealthier, more implicit, or adversarial SSOs that may be underrepresented in the current benchmark but are likely to become increasingly important as the skills ecosystem evolves.

w/o Neuro Reasoner. Removing the neuro reasoning component causes the largest overall degradation, reducing F1 from 0.93 to 0.71 and increasing FPR from 0.05 to 0.49. This shows that symbolic reasoning alone is insufficient for reliably distinguishing malicious from benign skills at the semantic level. In particular, although symbolic reasoning can capture suspicious operations, it is less effective at interpreting intent and context. For example, a pattern such as `curl | bash` may appear suspicious structurally, but its actual risk depends on contextual factors such as the source being fetched. Therefore, neuro reasoning is the key component for controlling FPs and making accurate decisions.

w/o Symbolic Reasoner. Finally, removing symbolic reasoning does not change the detection metrics on this benchmark, suggesting that neuro reasoning dominates the final classification performance in this setting. However, the main purpose of symbolic reasoning is not necessarily to improve benchmark accuracy in isolation. Instead, it provides structured intermediate constraints that can guide downstream reasoning, reduce unnecessary LLM involvement, and improve cost-effectiveness and reasoning stability in large-scale deployment. In our design, symbolic reasoning serves as an efficient first-stage filter. Although it may still produce FPs or miss some cases when used alone, these limitations can be mitigated through neuro-symbolic feedback to improve recall, while the subsequent neuro reasoning further suppresses FPs. Therefore, the symbolic-then-neural reasoning strategy is important not only for effectiveness, but also for scalable ecosystem-wide scanning.

Table 5: Performance of MalSkills with different LLMs.

LLM	TP/FP/TN/FN	Prec.	Rec.	FPR	F1
GPT-5.3-CodeX-Medium	92/5/95/8	**0.95**	**0.92**	**0.05**	**0.93**
Claude-Sonnet-4.6	70/14/86/30	0.83	0.70	0.14	0.76
Gemini-3.1-Flash-Lite	95/24/76/5	0.80	**0.95**	0.24	**0.87**
Qwen3.5-397B-A17B	78/15/85/22	0.84	0.78	0.15	0.81
DeepSeek-V3.2	71/5/95/29	**0.93**	0.71	**0.05**	0.81

5.4 RQ3: Impact of LLMs

To understand how the choice of LLM affects MalSkills, we run MalSkills with several widely used LLMs under the same experimental setting, including Claude Sonnet 4.6, Gemini 3.1 Flash-Lite, Qwen3.5-397B-A17B, and DeepSeek-V3.2. Table 5 reports the results. Overall, the choice of LLMs has a clear impact on the effectiveness of MalSkills, mainly by shifting the trade-off between recall and FP control. GPT-5.3-CodeX-Medium achieves the best overall performance, obtaining the highest precision (0.95), the lowest FPR (0.05, tied), and the best F1-score (0.93). This suggests that it provides the most balanced behavior across both extraction and reasoning, enabling MalSkills to identify malicious skills accurately without introducing excessive false alarms. Gemini-3.1-Flash-Lite achieves the highest recall (0.95), correctly identifying almost all malicious

Table 6: Distribution of the originally reported and manually confirmed malicious skills.

Source	Orig./Conf.	P1	P2	P3	P4	P5	P6
ClawHub [53]	252/230	204	16	8	-	1	1
Skills Directory [67]	97/25	20	3	1	1	-	-
Skills.sh [72]	11/7	7	-	-	-	-	-
SkillsLLM [68]	51/25	24	1	-	-	-	-
SkillsMP [69]	58/35	32	2	1	-	-	-
Skills.rest [70]	112/58	54	3	1	-	-	-
Smithery [73]	39/20	18	1	1	-	-	-
Total	**620/400**	**359**	**26**	**12**	**1**	**1**	**1**

Pattern Labels: P1: Data Theft/Exfiltration; P2: Instruction Override/Deception; P3: Remote Code Execution; P4: Remote Fetch/Supply Chain; P5: Privilege Escalation/Lateral Movement; P6: Stealth/Evasion/Obfuscation.

skills, but this comes at the cost of a noticeably higher FPR (0.24). This pattern suggests that Gemini tends to behave more aggressively, surfacing a larger set of potentially suspicious behaviors and therefore missing fewer malicious samples, but at the same time over-classifying more benign skills as malicious. DeepSeek-V3.2 exhibits the opposite behavior. It achieves very high precision (0.93) and the lowest FPR (0.05, tied with GPT), but its recall drops to 0.71. This indicates a more conservative decision boundary. Claude Sonnet 4.6 shows a similar but less favorable pattern, with both lower recall and lower overall F1. Qwen3.5-397B-A17B provides a more balanced middle ground, although it still falls short of GPT in both recall and FP control. These results indicate that MalSkills is robust across different model families, but the quality of the underlying LLM still matters. Stronger or more code- and instruction-reliable models can noticeably improve the end-to-end effectiveness.

5.5 RQ4: Practicality

Overall Results. To evaluate the real-world practicality of MalSkills, we applied it to *Wild-Skills-150K*. MalSkills initially flagged 620 potentially malicious skills. We then manually reviewed all reported cases. Specifically, the first two authors, who have 4 and 2 years of security experience, independently examined each reported skill and determined whether it exhibited malicious behavior. For cases with inconsistent conclusions, a third author with 7 years of security experience was introduced to arbitrate and assign the final label. The manual review confirms that 400 of the 620 flagged skills are indeed malicious. The resulting precision is lower than that on MalSkillsBench because RQ1 uses a balanced benchmark, whereas RQ4 operates in an open-world setting where malicious skills are rare. Although the FPR cannot be computed because TN remains unknown, the remaining 220 flagged skills correspond to nearly 0.15% of all 150,108 scanned skills. Many of these cases arise from benign skills that contain risky-looking patterns, such as remote installation behaviors (e.g., `curl | bash`) or credentialed access to remote endpoints. Determining whether such behaviors are truly malicious often requires inspecting second-stage payloads or richer execution contexts, and thus manual review remains necessary. As of now, all confirmed malicious skills have been responsibly reported and are currently awaiting confirmation from the platforms.
Registry Distribution. As shown in Table 6, the confirmed malicious skills are distributed across all 7 registries, with ClawHub contributing the largest portion. Notably, even registries that claim to incorporate security mechanisms still contained malicious skills. For example, ClawHub is protected by its own moderation engine [54] together with VirusTotal [55], while Skills.sh [71] integrates Gen Agent Trust Hub [21], Socket [34], and Snyk [14], and Skills Directory also claims security checks [66]. Despite these protections, we still identified 262 previously undisclosed malicious skills in these three registries. These findings suggest that currently deployed scanning mechanisms are still insufficient to reliably prevent malicious skills from entering public ecosystems.
Malicious Pattern. In terms of attack behaviors, data theft and exfiltration dominate the confirmed malicious skills (89.8%), followed by instruction override and deception, remote code execution, remote fetch or supply-chain abuse, privilege escalation, and stealth or obfuscation. Overall, these findings demonstrate that MalSkills is practical for ecosystem-scale scanning and effective at uncovering previously unknown malicious skills in real-world registries.

6 Discussion

Limitations. Our study has several limitations. First, although MalSkillsBench is constructed from real-world incidents, it may not fully cover the diversity of malicious skills in the wild. That said, we believe it still provides a realistic and reproducible foundation for comparison, especially given the current lack of public benchmarks. Second, while MalSkills supports multi-language parsing, deeper operand resolution currently depends on YASA [7, 82] and is limited to Python, JavaScript, Java, and Go. Nevertheless, this limitation is not fundamental, and future works can incorporate additional analyzers to improve the prototype. Third, as malicious-skill ecosystems evolve, attackers may adopt more implicit or adversarial strategies that reduce explicit security signals. Although MalSkills is intentionally designed to avoid direct maliciousness judgment and instead reason over extracted SSOs, it may still be affected by AI-evasion strategies [11, 48]. More broadly, malicious-skill detection remains an ongoing hide-and-seek between attackers and defenders, and we therefore view this work as a foundation for future research rather than a once-and-for-all solution.
Implications. Our findings suggest three broader implications for securing the emerging agentic supply chain. First, defenses for traditional software supply chains cannot be directly transferred to skill ecosystems. Unlike traditional packages, the risk of a skill often emerges from interactions among heterogeneous artifacts, so the defenses should treat cross-artifact reasoning as a basic requirement. Second, our results indicate that current protection mechanisms remain insufficient in practice. Although some registries have already integrated protections and the community has developed a range of scanning tools, our benchmark results and real-world findings show that these mechanisms still miss malicious skills or incur substantial false positives. Third, we believe that both the MalSkillsBench and the neuro-symbolic design of MalSkills can serve as a foundation for future research and inspire more effective defenses for these evolving threats.

7 Related Work

OSS Supply Chain Security. Open-source software (OSS) supply chain security has become a major research focus as attackers increasingly target package registries and dependency ecosystems. Prior works [23, 35] have systematically studied the attack surface of traditional OSS supply chains, including typosquatting [35, 83], dependency confusion [35], and even package confusion [50]. In response, researchers have proposed a wide range of detection and mitigation approaches. *Traditional methods* detect suspicious behaviors through semantic rules [38, 90], taint-style analysis [15, 38], and sandbox execution [15, 90]. *Learning-based methods* use metadata [25] and source-code features [36, 62] to train classifiers. More recently, *LLM-based approaches* have been explored for sensitive API identification [31], data augmentation [20], code understanding [80, 86], and maliciousness judgment [84, 88]. In contrast, MalSkills targets agent skills, whose malicious behavior is often distributed across artifacts rather than code scripts only.

Agentic Supply Chain Security. With the rise of LLM agents, recent work has started to examine security risks in the agentic supply chain [28, 30, 81], including agent skills [24, 41, 43] and MCP servers [27, 79]. Public reports and benchmark efforts have shown that real-world skill ecosystems already contain attacks such as wallet theft [4, 57] and reverse shells [75]. Preliminary defenses include rule-based scanners [22, 32], LLM-based analysis [12], and dynamic analysis [41] for skills. However, most existing tools analyze individual artifacts and provide limited support for cross-artifact reasoning. MalSkills addresses this gap by focusing on neuro-symbolic reasoning on heterogeneous artifacts.

Agent Security. Recent work has shown that LLM-based agents are highly vulnerable to taint-style and prompt-driven attacks. In particular, prior studies have demonstrated that attackers can manipulate agent inputs to steer model outputs into generating malicious code [40] or SQL queries [58], which leads to a series of injection vulnerabilities. In response, recent efforts have proposed analysis techniques for detecting such vulnerabilities, including static taint analysis [26] and greybox fuzzing [39, 40]. In comparison, our work considers a complementary threat model in which adversaries inject malicious skills into the agent ecosystem and compromise system security.

8 Conclusion

In this paper, we presented MalSkills, a neuro-symbolic framework for detecting malicious skills in the emerging agentic supply chain. By combining cross-artifact SSO extraction, skill dependency graph construction, and neuro-symbolic reasoning, MalSkills is designed to better handle the heterogeneous, context-dependent, and adversarial nature of malicious skills. Our evaluation on MalSkills-Bench shows that MalSkills achieves the best overall detection performance among representative baselines, while our large-scale deployment on 7 public registries further demonstrates its practicality and ability to uncover previously unknown malicious skills in the wild. Overall, we hope this work can serve as a foundation for future research on securing the agentic supply chain.

Acknowledgements

We used GenAI tools for language refinement. We reviewed all AI-assisted content and take full responsibility for this paper. This work was supported in part by the National Natural Science Foundation of China (grants No.62572209, 62502168), the Hubei Provincial Key Research and Development Program (grant No. 2025BAB057), and Ant Group Research Fund.

Data Availability Statement

We have made the implementations of MalSkills and experimental data publicly accessible at https://github.com/security-pride/MalSkills.

References

[1] A2A Protocol. [n. d.]. A2A Protocol. https://a2a-protocol.org/. Accessed: 2026-03-25.
[2] Agency. [n. d.]. Agents Marketplace. https://marketplace.agen.cy/agents. Accessed: 2026-03-25.
[3] Agent Skills. 2026. Agent Skills. https://agentskills.io/. Accessed: 2026-03-20.
[4] Alex and Oren Yomtov. 2026. ClawHavoc: 341 Malicious Clawed Skills Found by the Bot They Were Targeting. https://www.koi.ai/blog/clawhavoc-341-malicious-clawedbot-skills-found-by-the-bot-they-were-targeting. Accessed: 2026-03-21.
[5] Alice. 2026. Caterpillar: Security Scanner for AI Agent Skill Files. https://github.com/alice-dot-io/caterpillar. Accessed 2026-03-25.
[6] Hyrum S. Anderson and Phil Roth. 2018. EMBER: An Open Dataset for Training Static PE Malware Machine Learning Models. *CoRR* abs/1804.04637 (2018). arXiv:1804.04637 http://arxiv.org/abs/1804.04637
[7] Ant Group. 2026. YASA-Engine. https://github.com/antgroup/YASA-Engine. Accessed 2026-03-26.
[8] Aryaman Behera. 2026. AI Agent Skill Scanners: Every Tool Compared (2026). https://repello.ai/blog/ai-agent-skill-scanner. Repello AI blog post, accessed 2026-03-22.
[9] CCCS-CIC. 2020. CCCS-CIC-AndMal-2020 Dataset. https://www.unb.ca/cic/datasets/andmal2020.html. Accessed 2026-03-26.
[10] CertiK. 2026. Skill Scanning Is Not a Security Boundary. https://www.certik.com/blog/skill-scanning-is-not-a-security-boundary. Industry Research blog post, accessed 2026-03-22.
[11] Check Point Research. 2025. AI Evasion: The Next Frontier of Malware Techniques. https://blog.checkpoint.com/artificial-intelligence/ai-evasion-the-next-frontier-of-malware-techniques/. Check Point Blog, accessed 2026-03-22.
[12] Cisco AI Defense. 2026. Skill-Scanner: Security Scanner for Agent Skills. https://github.com/cisco-ai-defense/skill-scanner. Accessed: 2026-03-21.
[13] CYFIRMA. 2026. Malware Detection Evasion Techniques. https://www.cyfirma.com/research/malware-detection-evasion-techniques/. Accessed: 2026-03-22.
[14] Degges, Randall. 2026. Securing the Agent Skill Ecosystem: How Snyk and Vercel Are Locking Down the New Software Supply Chain. https://snyk.io/blog/snyk-vercel-securing-agent-skill-ecosystem/ Accessed: 2026-03-27.
[15] Ruian Duan, Omar Alrawi, Ranjita Pai Kasturi, Ryan Elder, Brendan Saltaformaggio, and Wenke Lee. 2021. Towards Measuring Supply Chain Attacks on Package Managers for Interpreted Languages. In *28th Annual Network and Distributed System Security Symposium, NDSS*. https://www.ndss-symposium.org/wp-content/uploads/ndss2021_1B-1_23055_paper.pdf
[16] Shehan Edirimannage, Charitha Elvitigala, Asitha Kottahachchi Kankanamge Don, Wathsara Daluwatta, Primal Wijesekara, and Ibrahim Khalil. 2024. Developers Are Victims Too : A Comprehensive Analysis of The VS Code Extension Ecosystem. *CoRR* abs/2411.07479 (2024). https://doi.org/10.48550/ARXIV.2411.07479 arXiv:2411.07479
[17] Brandon Edwards and Nick Freeman. 2019. A Compendium of Container Escapes. https://i.blackhat.com/USA-19/Thursday/us-19-Edwards-Compendium-Of-Container-Escapes-up.pdf. Accessed 2026-03-22.
[18] Arthur Le Faou. 2024. Antivirus and EDR Bypass Techniques. https://www.vaadata.com/blog/antivirus-and-edr-bypass-techniques/. Accessed 2026-03-22.
[19] Fortinet. 2026. Living Off The Land (LOTL) Attacks and Techniques. https://www.fortinet.com/resources/cyberglossary/living-off-the-land-lotl. Accessed: 2026-03-22.
[20] Xingan Gao, Xiaobing Sun, Sicong Cao, Kaifeng Huang, Di Wu, Xiaolei Liu, Xingwei Lin, and Yang Xiang. 2025. MalGuard: Towards Real-Time, Accurate, and Actionable Detection of Malicious Packages in PyPI Ecosystem. *CoRR* abs/2506.14466 (2025). https://doi.org/10.48550/ARXIV.2506.14466 arXiv:2506.14466
[21] Gen Digital. 2026. AI by Gen: Skill Scanner. https://ai.gendigital.com/skill-scanner Accessed: 2026-03-27.

[22] GoPlus Security. 2026. AgentGuard: Security Guard for AI Agents. https://github.com/GoPlusSecurity/agentguard. Accessed: 2026-03-21.

[23] Yacong Gu, Lingyun Ying, Yingyuan Pu, Xiao Hu, Huajun Chai, Ruimin Wang, Xing Gao, and Haixin Duan. 2023. Investigating Package Related Security Threats in Software Registries. In *44th IEEE Symposium on Security and Privacy, SP 2023, San Francisco, CA, USA, May 21-25, 2023*. IEEE, 1578–1595. https://doi.org/10.1109/SP46215.2023.10179332

[24] Zihan Guo, Zhiyu Chen, Xiaohang Nie, Jianghao Lin, Yuanjian Zhou, and Weinan Zhang. 2026. SkillProbe: Security Auditing for Emerging Agent Skill Marketplaces via Multi-Agent Collaboration. arXiv:2603.21019 [cs.CR] https://arxiv.org/abs/2603.21019

[25] Sajal Halder, Michael Bewong, Arash Mahboubi, Yinhao Jiang, Md. Rafiqul Islam, Md Zahidul Islam, Ryan H. L. Ip, Muhammad Ejaz Ahmed, Gowri Sankar Ramachandran, and Muhammad Ali Babar. 2024. Malicious Package Detection using Metadata Information. In *Proceedings of the ACM on Web Conference 2024, WWW 2024, Singapore, May 13-17, 2024*, Tat-Seng Chua, Chong-Wah Ngo, Ravi Kumar, Hady W. Lauw, and Roy Ka-Wei Lee (Eds.). ACM, 1779–1789. https://doi.org/10.1145/3589334.3645543

[26] Junjie He, Shenao Wang, Yanjie Zhao, Xinyi Hou, Zhao Liu, Quanchen Zou, and Haoyu Wang. 2026. TaintP2X: Detecting Taint-Style Prompt-to-Anything Injection Vulnerabilities in LLM-Integrated Applications. In *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering*. https://doi.org/10.1145/3744916.3773199

[27] Xinyi Hou, Yanjie Zhao, Shenao Wang, and Haoyu Wang. 2026. Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions. *ACM Trans. Softw. Eng. Methodol.* (Feb. 2026). https://doi.org/10.1145/3796519 Just Accepted.

[28] Qiang Hu, Xiaofei Xie, Sen Chen, Lili Quan, and Lei Ma. 2025. Large Language Model Supply Chain: Open Problems From the Security Perspective. In *Proceedings of the 34th ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA Companion 2025, Clarion Hotel Trondheim, Trondheim, Norway, June 25-28, 2025*, Mike Papadakis, Myra B. Cohen, and Paolo Tonella (Eds.). ACM, 169–173. https://doi.org/10.1145/3713081.3731747

[29] Cheng Huang, Nannan Wang, Ziyan Wang, Siqi Sun, Lingzi Li, Junren Chen, Qianchong Zhao, Jiaxuan Han, Zhen Yang, and Lei Shi. 2024. DONAPI: Malicious NPM Packages Detector using Behavior Sequence Knowledge Mapping. In *33rd USENIX Security Symposium, USENIX Security 2024, Philadelphia, PA, USA, August 14-16, 2024*, Davide Balzarotti and Wenyuan Xu (Eds.). USENIX Association. https://www.usenix.org/conference/usenixsecurity24/presentation/huang-cheng

[30] Kaifeng Huang, Bihuan Chen, You Lu, Susheng Wu, Dingji Wang, Yiheng Huang, Haowen Jiang, Zhuotong Zhou, Junming Cao, and Xin Peng. 2025. Lifting the Veil on Composition, Risks, and Mitigations of the Large Language Model Supply Chain. arXiv:2410.21218 [cs.SE] https://arxiv.org/abs/2410.21218

[31] Yiheng Huang, Ruisi Wang, Wen Zheng, Zhuotong Zhou, Susheng Wu, Shulin Ke, Bihuan Chen, Shan Gao, and Xin Peng. 2024. SpiderScan: Practical Detection of Malicious NPM Packages Based on Graph-Based Behavior Modeling and Matching. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, ASE 2024, Sacramento, CA, USA, October 27 - November 1, 2024*, Vladimir Filkov, Baishakhi Ray, and Minghui Zhou (Eds.). ACM, 1146–1158. https://doi.org/10.1145/3691620.3695492

[32] Huifer. 2026. Skill-Security-Scan. https://github.com/huifer/skill-security-scan. Accessed: 2026-03-21.

[33] Zhiqiang Jian and Long Chen. 2017. A Defense Method against Docker Escape Attack. In *Proceedings of the 2017 International Conference on Cryptography, Security and Privacy* (Wuhan, China) *(ICCSP '17)*. Association for Computing Machinery, New York, NY, USA, 142–146. https://doi.org/10.1145/3058060.3058085

[34] Jiang, Wenxin and Kapravelos, Alexandros. 2026. Socket Brings Supply Chain Security to skills.sh. https://socket.dev/blog/socket-brings-supply-chain-security-to-skills Accessed: 2026-03-27.

[35] Piergiorgio Ladisa, Henrik Plate, Matias Martinez, and Olivier Barais. 2023. SoK: Taxonomy of Attacks on Open-Source Software Supply Chains. In *2023 IEEE Symposium on Security and Privacy (SP)*. 1509–1526. https://doi.org/10.1109/SP46215.2023.10179304

[36] Piergiorgio Ladisa, Serena Elisa Ponta, Nicola Ronzoni, Matias Martinez, and Olivier Barais. 2023. On the Feasibility of Cross-Language Detection of Malicious Packages in npm and PyPI. In *Annual Computer Security Applications Conference, ACSAC 2023, Austin, TX, USA, December 4-8, 2023*. ACM, 71–82. https://doi.org/10.1145/3627106.3627138

[37] LangChain. 2026. LangChain Skills. https://docs.langchain.com/oss/python/deepagents/skills. Accessed: 2026-03-20.

[38] Ningke Li, Shenao Wang, Mingxi Feng, Kailong Wang, Meizhen Wang, and Haoyu Wang. 2023. MalWuKong: Towards Fast, Accurate, and Multilingual Detection of Malicious Code Poisoning in OSS Supply Chains. In *38th IEEE/ACM International Conference on Automated Software Engineering, ASE 2023, Luxembourg, September 11-15, 2023*. IEEE, 1993–2005. https://doi.org/10.1109/ASE56229.2023.00073

[39] Fengyu Liu, Yuan Zhang, Jiaqi Luo, Jiarun Dai, Tian Chen, Letian Yuan, Zhengmin Yu, Youkun Shi, Ke Li, Chengyuan Zhou, Hao Chen, and Min Yang. 2025. Make Agent Defeat Agent: Automatic Detection of Taint-Style Vulnerabilities in LLM-based Agents. In *34th USENIX Security Symposium, USENIX Security 2025, Seattle, WA, USA, August 13-15, 2025*, Lujo Bauer and Giancarlo Pellegrino (Eds.). USENIX Association, 3767–3786. https://www.usenix.org/conference/usenixsecurity25/presentation/liu-fengyu

[40] Tong Liu, Zizhuang Deng, Guozhu Meng, Yuekang Li, and Kai Chen. 2024. Demystifying RCE Vulnerabilities in LLM-Integrated Apps. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security* (Salt Lake City, UT, USA) *(CCS '24)*. Association for Computing Machinery, New York, NY, USA, 1716–1730. https://doi.org/10.1145/3658644.3690338

[41] Yi Liu, Zhihao Chen, Yanjun Zhang, Gelei Deng, Yuekang Li, Jianting Ning, Ying Zhang, and Leo Yu Zhang. 2026. Malicious Agent Skills in the Wild: A Large-Scale Security Empirical Study. arXiv:2602.06547 [cs.CR] https://arxiv.org/abs/2602.06547

[42] Yue Liu, Chakkrit Tantithamthavorn, and Li Li. 2025. Protect Your Secrets: Understanding and Measuring Data Exposure in VSCode Extensions. In *2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. 551–562. https://doi.org/10.1109/SANER64311.2025.00058

[43] Yi Liu, Weizhe Wang, Ruitao Feng, Yao Zhang, Guangquan Xu, Gelei Deng, Yuekang Li, and Leo Zhang. 2026. Agent Skills in the Wild: An Empirical Study of Security Vulnerabilities at Scale. arXiv:2601.10338 [cs.CR] https://arxiv.org/abs/2601.10338

[44] LobeHub. [n. d.]. Agent Marketplace. https://lobehub.com/agent. Accessed: 2026-03-25.

[45] LOLBAS Project. 2026. LOLBAS: Living Off The Land Binaries, Scripts and Libraries. https://lolbas-project.github.io/. Accessed: 2026-03-22.

[46] MCP Market. [n. d.]. MCP Market. https://mcpmarket.com/. Accessed: 2026-03-25.

[47] mcp.so. [n. d.]. mcp.so. https://mcp.so/. Accessed: 2026-03-25.

[48] Joey Melo. 2025. Bypassing AI Malware Analysis with Prompt Injection. https://pangea.cloud/blog/bypassing-ai-malware-analysis-with-prompt-injection/. Accessed 2026-03-22.

[49] Model Context Protocol. [n. d.]. Model Context Protocol. https://modelcontextprotocol.io/docs/getting-started/intro. Accessed: 2026-03-25.

[50] Shradha Neupane, Grant Holmes, Elizabeth Wyss, Drew Davidson, and Lorenzo De Carli. 2023. Beyond Typosquatting: An In-depth Look at Package Confusion. In *32nd USENIX Security Symposium (USENIX Security 23)*. USENIX Association, Anaheim, CA, 3439–3456. https://www.usenix.org/conference/usenixsecurity23/presentation/neupane

[51] OpenAI. 2026. OpenAI Agent Skills. https://developers.openai.com/codex/skills. Accessed: 2026-03-20.

[52] openclaw. 2026. All versions of all skills that are on clawhub archived. https://github.com/openclaw/skills. Accessed 2026-03-26.

[53] OpenClaw. 2026. ClawHub. https://clawhub.ai/. Accessed: 2026-03-20.

[54] OpenClaw. 2026. moderationEngine. https://github.com/openclaw/clawhub/blob/6318a74adff5bb01c2e503e9226f86cf68770e09/convex/lib/moderationEngine.ts. Accessed: 2026-03-21.

[55] OpenClaw. 2026. OpenClaw Partners with VirusTotal to Improve Skill Security. https://openclaw.ai/blog/virustotal-partnership. Accessed: 2026-03-21.

[56] OpenClaw. 2026. OpenClaw Skills. https://docs.openclaw.ai/tools/skills. Accessed: 2026-03-20.

[57] OpenSourceMalware. 2026. Clawdbot Skills Ganked Your Crypto. https://opensourcemalware.com/blog/clawdbot-skills-ganked-your-crypto. Accessed 2026-03-26.

[58] Rodrigo Pedro, Miguel E. Coimbra, Daniel Castro, Paulo Carreira, and Nuno Santos. 2025. Prompt-to-SQL Injections in LLM-Integrated Web Applications: Risks and Defenses . In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, Los Alamitos, CA, USA, 76–88. https://doi.org/10.1109/ICSE55347.2025.00007

[59] philskents. 2026. clawhub-blocklist. https://github.com/philskents/clawhub-blocklist. Accessed 2026-03-26.

[60] PiedPiper0709. 2026. openclaw-malicious-skills. https://github.com/PiedPiper0709/openclaw-malicious-skills. Accessed 2026-03-26.

[61] Hongzhou Rao, Zikan Dong, Yanjie Zhao, Haodong Li, and Haoyu Wang. 2026. ACODA: Adversarial Code Obfuscation for Defending against LLM-based Analysis. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE-Companion)*.

[62] Adriana Sejfia and Max Schäfer. 2022. Practical Automated Detection of Malicious npm Packages. In *44th IEEE/ACM 44th International Conference on Software Engineering, ICSE 2022, Pittsburgh, PA, USA, May 25-27, 2022*. ACM, 1681–1692. https://doi.org/10.1145/3510003.3510104

[63] Semgrep. 2026. Semgrep. https://github.com/semgrep/semgrep. Accessed 2026-03-26.

[64] Semgrep. 2026. Semgrep Community Edition rules. https://github.com/semgrep/semgrep-rules. Accessed 2026-03-26.

[65] sggolakiya. 2026. ClawScan: Security scanner for OpenClaw skills. https://github.com/sggolakiya/clawscan. Accessed March 22, 2026.

[66] Skills Directory. 2026. Security Mechanism of Skills Directory. https://www.skillsdirectory.com/security Accessed: 2026-03-27.

[67] Skills Directory. 2026. Skills Directory. https://www.skillsdirectory.com/. Accessed 2026-03-26.

[68] SkillsLLM. 2026. SkillsLLM. https://skillsllm.com/. Accessed 2026-03-26.

[69] SkillsMP. 2026. Agent Skills Marketplace. https://skillsmp.com/. Accessed: 2026-03-20.

[70] skills.rest. 2026. Agent Skills Library. https://skills.rest/ Accessed: 2026-03-27.

[71] Skills.sh. 2026. Security Audits of Skills.sh. https://skills.sh/audits Accessed: 2026-03-27.

[72] Skills.sh. 2026. Skills.sh. https://skills.sh/. Accessed: 2026-03-20.

[73] Smithery. 2026. Smithery. https://smithery.ai/. Accessed 2026-03-26.

[74] Snyk. 2026. Agent Scan: Security Scanner for AI Agent Supply Chains. https://github.com/snyk/agent-scan. Accessed: 2026-03-21.

[75] Snyk Security. 2026. Snyk Finds Prompt Injection in 36%, 1467 Malicious Payloads in a ToxicSkills Study of Agent Skills Supply Chain Compromise. https://snyk.io/de/blog/toxicskills-malicious-ai-agent-skills-clawhub/. Accessed: 2026-03-21.

[76] Xiaobing Sun, Xingan Gao, Sicong Cao, Lili Bo, Xiaoxue Wu, and Kaifeng Huang. 2024. 1+1>2: Integrating Deep Code Behaviors with Metadata Features for Malicious PyPI Package Detection. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, ASE 2024, Sacramento, CA, USA, October 27 - November 1, 2024*, Vladimir Filkov, Baishakhi Ray, and Minghui Zhou (Eds.). ACM, 1159–1170. https://doi.org/10.1145/3691620.3695493

[77] Liran Tal. 2026. Why Your "Skill Scanner" Is Just False Security (and Maybe Malware). https://snyk.io/blog/skill-scanner-false-security/. Accessed 2026-03-22.

[78] Thomas Roccia. 2026. Nova-Proximity: A MCP and Agent Skills Security Scanner Powered with NOVA. https://github.com/Nova-Hunting/nova-proximity. Accessed 2026-03-25.

[79] Bin Wang, Zexin Liu, Hao Yu, Ao Yang, Yenan Huang, Jing Guo, Huangsheng Cheng, Hui Li, and Huiyu Wu. 2025. MCPGuard : Automatically Detecting Vulnerabilities in MCP Servers. arXiv:2510.23673 [cs.CR] https://arxiv.org/abs/2510.23673

[80] Jian Wang, Zhen Li, Jixiang Qu, Deqing Zou, Shouhuai Xu, Ziteng Xu, Zhenwei Wang, and Hai Jin. 2025. MalPacDetector: An LLM-Based Malicious NPM Package Detector. *IEEE Trans. Inf. Forensics Secur.* 20 (2025), 6279–6291. https://doi.org/10.1109/TIFS.2025.3580336

[81] Shenao Wang, Yanjie Zhao, Xinyi Hou, and Haoyu Wang. 2025. Large Language Model Supply Chain: A Research Agenda. *ACM Trans. Softw. Eng. Methodol.* 34, 5 (2025), 147:1–147:46. https://doi.org/10.1145/3708531

[82] Yayi Wang, Shenao Wang, Jian Zhao, Shaosen Shi, Ting Li, Yan Cheng, Lizhong Bian, Kan Yu, Yanjie Zhao, and Haoyu Wang. 2026. YASA: Scalable Multi-Language Taint Analysis on the Unified AST at Ant Group. arXiv:2601.17390 [cs.SE] https://arxiv.org/abs/2601.17390

[83] Yinglin Xie, Xinyi Hou, Yanjie Zhao, Kai Chen, and Haoyu Wang. 2025. *LLM App Squatting and Cloning*. Association for Computing Machinery, New York, NY, USA, 64–74. https://doi.org/10.1145/3696630.3728532

[84] Di Xue, Gang Zhao, Zhongqi Fan, Wei Li, Yahong Xu, Zhen Liu, Yin Liu, and Zhongliang Yuan. 2024. Poster: An Exploration of Large Language Models in Malicious Source Code Detection. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS 2024, Salt Lake City, UT, USA, October 14-18, 2024*, Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie (Eds.). ACM, 4940–4942. https://doi.org/10.1145/3658644.3691374

[85] Yara-Rules. 2026. Repository of YARA rules. https://github.com/yara-rules/rules. Accessed 2026-03-26.

[86] Zeliang Yu, Ming Wen, Xiaochen Guo, and Hai Jin. 2024. Maltracker: A Fine-Grained NPM Malware Tracker Copiloted by LLM-Enhanced Dataset. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024, Vienna, Austria, September 16-20, 2024*, Maria Christakis and Michael Pradel (Eds.). ACM, 1759–1771. https://doi.org/10.1145/3650212.3680397

[87] Nusrat Zahan, Philipp Burckhardt, Mikola Lysenko, Feross Aboukhadijeh, and Laurie Williams. 2024. MalwareBench: Malware samples are not enough. In *Proceedings of the 21st International Conference on Mining Software Repositories* (Lisbon, Portugal) *(MSR '24)*. Association for Computing Machinery, New York, NY, USA, 728–732. https://doi.org/10.1145/3643991.3644883

[88] Nusrat Zahan, Philipp Burckhardt, Mikola Lysenko, Feross Aboukhadijeh, and Laurie A. Williams. 2025. Leveraging Large Language Models to Detect NPM Malicious Packages. In *47th IEEE/ACM International Conference on Software Engineering, ICSE 2025, Ottawa, ON, Canada, April 26 - May 6, 2025*. IEEE, 2625–2637. https://doi.org/10.1109/ICSE55347.2025.00146

[89] Junan Zhang, Kaifeng Huang, Yiheng Huang, Bihuan Chen, Ruisi Wang, Chong Wang, and Xin Peng. 2025. Killing Two Birds with One Stone: Malicious Package Detection in NPM and PyPI using a Single Model of Malicious Behavior Sequence. *ACM Trans. Softw. Eng. Methodol.* 34, 4, Article 104 (April 2025), 28 pages. https://doi.org/10.1145/3705304

[90] Xinyi Zheng, Chen Wei, Shenao Wang, Yanjie Zhao, Peiming Gao, Yuanchao Zhang, Kailong Wang, and Haoyu Wang. 2024. Towards Robust Detection of Open Source Software Supply Chain Poisoning Attacks in Industry Environments. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, ASE 2024, Sacramento, CA, USA, October 27 - November 1, 2024*, Vladimir Filkov, Baishakhi Ray, and Minghui Zhou (Eds.). ACM, 1990–2001. https://doi.org/10.1145/3691620.3695262